API Gateway Pattern

one front door, no business logic

Watch the 4-min video

In one line: an API gateway is the single entry point for external clients: it routes requests, aggregates calls and handles cross-cutting concerns (TLS, auth, rate limits, logging), but business logic never lives in it.

1 Twenty addresses for one app

A mobile app talks to twenty microservices directly. It needs twenty hostnames, each service re-implements authentication, and one screen takes five round trips over a slow mobile network. When a service is split or moved, old app versions in the wild break. An API gateway puts one front door in front of everything.

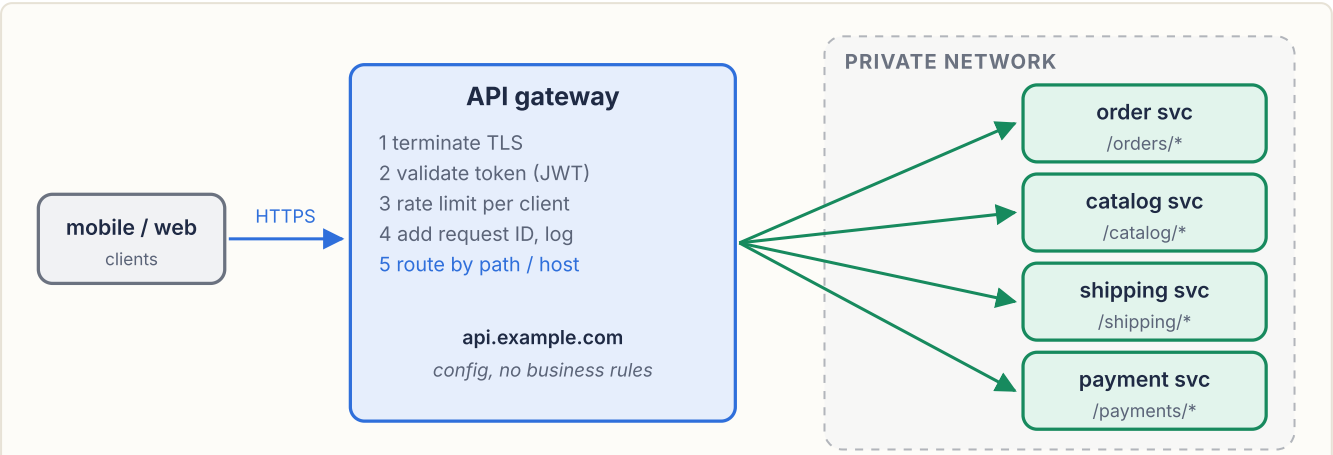


Figure 1. Every request passes the same checks once, then is routed by path to a private service.

2 Three jobs

1. Gateway routing. Clients call one public endpoint; the gateway routes by path, host or header (`/orders/*` → order service). Services can move, split or scale without clients noticing; the internal topology is hidden.

2. Gateway aggregation. One client request fans out to several services in parallel and the results are merged. The client makes one round trip instead of five, which matters on mobile networks with 100 ms+ round-trip times.

3. Gateway offloading. Cross-cutting concerns are done once: TLS termination, token validation, rate limiting and quotas, request IDs, access logs, CORS, response caching, request size limits. Products such as Kong, NGINX, Envoy, AWS API Gateway and Azure API Management provide these as configuration.

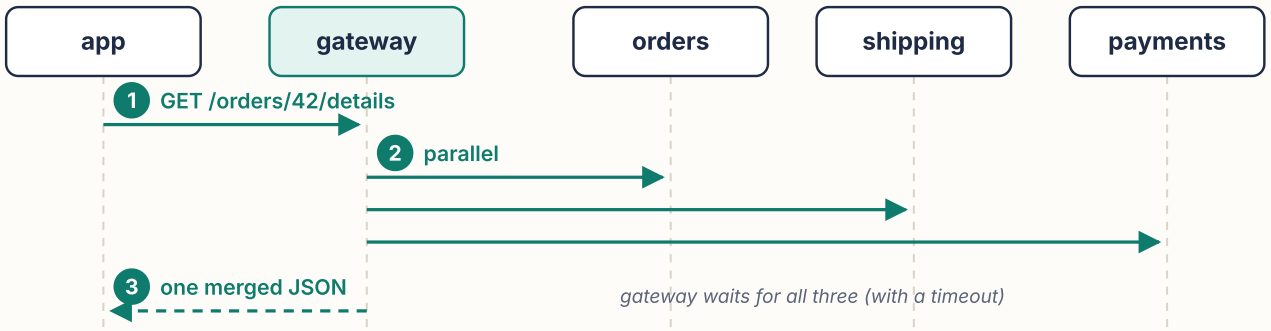


Figure 2. Aggregation: the slow part (several calls) happens inside the data centre, not over the mobile network.

3 What must never live in the gateway

Business logic. Pricing rules, discount checks, order workflows and data decisions belong in services owned by domain teams. Once they creep into the gateway it becomes a **shared monolith**: every team must change it, one team owns it, and it slows everyone down. A good gateway is boring: routing tables and policies in version control, not code full of rules. Note also that validating a token at the edge is **authentication**; whether this user may see order 42 is **authorisation** and stays in the order service.

Belongs in the gateway	Belongs in services
routing, load balancing, retries to healthy instances	domain rules: pricing, discounts, eligibility
TLS, token signature/expiry check	fine-grained authorisation (may user X see order 42?)
rate limits, quotas, request size limits	workflows and state changes
request IDs, access logs, metrics	validation of business data

4 Critical infrastructure

The gateway sits on every request, so it must be highly available (several instances across zones behind a load balancer), scaled out, monitored and changed carefully. Avoid one giant gateway owned by a bottleneck team: some companies run one gateway per domain or per client type, which leads to the BFF pattern (lesson 26).

WORKED EXAMPLE: TWENTY SERVICES, ONE ENDPOINT

All traffic enters through `api.example.com`. The gateway terminates TLS, checks the JWT's signature and expiry, applies a rate limit of, say, 100 requests per minute per client key (returning HTTP 429 when exceeded), adds an `X-Request-ID`, and routes by path. For `GET /orders/42/details` it calls orders, shipping and payments in parallel and returns one response. The twenty services have no public IPs; only the gateway is exposed to the internet.

COMMON MISTAKES

- Putting discount or workflow logic in gateway plugins or scripts.
- Treating gateway authentication as full authorisation.
- A single gateway instance or zone: a single point of failure.
- Aggregation with no per-call timeout, so one slow service delays every response.

5 Key terms

API gateway: single entry point for external clients

Gateway aggregation: fan out to several services, merge results

TLS termination: decrypt HTTPS at the gateway

Gateway routing: map path/host to an internal service

Gateway offloading: TLS, auth, rate limiting done once at the edge

HTTP 429: Too Many Requests: rate limit hit

6 Interview questions

Q1. What are the main jobs of an API gateway? BASIC

Routing requests to the right service, aggregating several calls into one response, and offloading cross-cutting concerns such as TLS, authentication, rate limiting and logging.

Q2. Why not let clients call services directly? BASIC

Clients would need every address and auth scheme, make many round trips, and break whenever services move; services would all re-implement the same edge concerns.

Q3. What logic should never live in the API gateway? SCENARIO

Business logic: domain rules, pricing, workflows and data decisions belong in team-owned services; otherwise the gateway becomes a shared monolith and bottleneck.

Q4. The order-details screen needs three services. How does the gateway help? SCENARIO

It calls orders, shipping and payments in parallel with timeouts and returns one merged response, so the client makes one round trip.

Q5. The gateway validates the JWT. Can the order service skip all checks? TRICKY

No: the gateway proves who the caller is, but the service must still check what they may do, e.g. that order 42 belongs to this user.

Q6. Isn't the gateway a single point of failure? TRICKY

It is a critical dependency, so run several stateless instances across zones behind a load balancer and keep it simple; splitting gateways per domain also limits blast radius.

Next: SD-L-008 · Backend for Frontend backendroadmap.com